

Detaillierte Aufgabenstellung

Allgemeine Angaben

Titel der Arbeit

Konzeption und Entwicklung eines Angular-Frontends zur Visualisierung von EFZ-Bildungsplandaten

Ausgangslage

Die Bildungspläne (EFZ) enthalten zentrale Informationen zu Handlungskompetenzbereichen, Handlungskompetenzen und Modulen. Diese Daten liegen elektronisch vor und werden über eine REST-API bereitgestellt.

Aktuell fehlt jedoch eine benutzerfreundliche Oberfläche, welche diese Informationen übersichtlich darstellt und zugänglich macht. Praxisausbildner und Lernende müssen daher auf wenig strukturierte Dokumente zurückgreifen. Dies erschwert den Zugriff auf relevante Inhalte und führt zu erhöhtem Zeitaufwand bei der Informationssuche.

Eine moderne Weboberfläche kann die vorhandenen Daten zentral, verständlich und effizient visualisieren. Durch ein Angular-basiertes UI soll eine klare Navigation zwischen Handlungskompetenzbereichen, Handlungskompetenzen und zugehörigen Inhalten ermöglicht werden. Zusätzlich sollen fortgeschrittene Suchfunktionen zur Verfügung stehen, um Inhalte gezielt zu filtern und schneller zu finden.

Ziel ist es, die bestehenden Daten aus der REST-API strukturiert aufzubereiten und in einer intuitiven Benutzeroberfläche darzustellen. Das Projekt ist ein Schritt in Richtung einer Applikation, welche Praxisausbildner und Lernende in der Planung der Praxisausbildung unterstützt.

Die Lösung soll künftig helfen, relevante Handlungskompetenzen und weitere Inhalte schneller zu identifizieren und in den Ausbildungsalltag zu integrieren. Dadurch wird die Planung strukturierter, transparenter und nachvollziehbarer.

Detaillierte Aufgabenstellung

Im Rahmen dieser Arbeit wird ein webbasiertes Angular-Frontend entwickelt, das Daten aus einer bestehenden REST-API zu EFZ-Bildungsplänen visualisiert. Der Fokus liegt auf der Darstellung und Durchsuchbarkeit von Handlungskompetenzbereichen, Handlungskompetenzen, Modulen und weiteren zugehörigen Inhalten. Ziel ist es, eine funktionale und übersichtliche Benutzeroberfläche zu erstellen, die als Grundlage für eine spätere Applikation zur Unterstützung der Praxisausbildungsplanung dient.

Zu Beginn erfolgt die Analyse der bestehenden REST-API sowie der verfügbaren Datenstrukturen. Die Beziehungen zwischen Handlungskompetenzbereichen, Handlungskompetenzen und Modulen werden nicht überall als vollständig verschachtelte Hierarchie geliefert. Stattdessen stellt die API einzelne Objekte sowie Referenzen bzw. IDs zur Verfügung, über die die Beziehungen im Frontend aufgelöst werden.

Das Angular-Frontend übernimmt somit die Aufgabe, Zusammenhänge zwischen den Entitäten über gezielte Service-Aufrufe herzustellen. Sämtliche Funktionen erfolgen im Kontext eines ausgewählten EFZ sowie – falls vorhanden – einer Spezialisierung. Alle angezeigten Daten und Suchresultate beziehen sich auf diese Auswahl.

Auf dieser Grundlage werden über List-Services passende Einträge geladen und in der Benutzeroberfläche zur Auswahl angeboten. Nach der Auswahl eines Eintrags werden die entsprechenden Detaildaten sowie bei Bedarf verknüpfte Objekte über weitere API-Calls geladen und angezeigt.

Beispielsweise können nach Auswahl eines Handlungskompetenzbereichs die zugehörigen Handlungskompetenzen geladen werden. Ausgehend von einer Handlungskompetenz können wiederum die zugehörigen Module dargestellt werden. Umgekehrt ist bei einem ausgewählten Modul ersichtlich, welchen Handlungskompetenzen dieses zugeordnet ist, sodass auch eine Navigation zurück zu übergeordneten Objekten möglich ist.

Dadurch entsteht eine schrittweise Beziehungsdarstellung, ohne dass alle Daten eines EFZ vorgängig geladen werden müssen. Darauf aufbauend wird ein geeignetes UI-Konzept erstellt, das eine klare Navigation und eine verständliche Darstellung der Inhalte ermöglicht.

Die Benutzeroberfläche stellt zentrale Informationen strukturiert dar und ermöglicht eine gezielte Suche. Dabei werden grundlegende Usability-Aspekte sowie eine nachvollziehbare Informationsarchitektur berücksichtigt.

Im Angular-Frontend werden folgende Funktionen umgesetzt:

- Exakte Suche nach Handlungskompetenzbereichen über ID aus List-API-Call mit Dropdown-Auswahl
- Anzeige eines ausgewählten Handlungskompetenzbereichs mit Detailinformationen
- Exakte Suche nach Handlungskompetenzen über ID aus List-API-Call mit Dropdown-Auswahl
- Anzeige einer ausgewählten Handlungskompetenz
- Anzeige der zu einer Handlungskompetenz gehörenden Module
- Exakte Suche nach Modulen über ID aus List-API-Call mit Dropdown-Auswahl
- Volltextsuche über Handlungskompetenzbereiche, Handlungskompetenzen und Module hinweg

Die exakten Suchfunktionen basieren auf List-API-Endpunkten, welche alle Einträge mit einer ID zurückliefern. Die Resultate werden in Dropdown-Komponenten dargestellt, aus denen ein konkreter Eintrag ausgewählt werden kann.

Die REST-API unterstützt keine generische Volltextsuche über alle Business-Objekte hinweg. Eine serverseitige Suche nach Stichworten ist lediglich für Module verfügbar. Für Handlungskompetenzbereiche und Handlungskompetenzen stehen ausschliesslich List-Endpunkte ohne Suchparameter zur Verfügung. Die Volltextsuche wird daher im Frontend umgesetzt, wo es keine Möglichkeit über die API gibt.

Die Daten werden über die entsprechenden Endpunkte geladen, clientseitig gefiltert und anschliessend zu einer einheitlichen Ergebnisliste aggregiert. Die Filterung erfolgt über relevante Textfelder (Bezeichnungen und Beschreibungen). Treffer sind nach Objekttyp gekennzeichnet und führen direkt zur Detailansicht.

In den Detailansichten werden sämtliche verfügbaren Informationen der Business-Objekte dargestellt. In den Übersichtsansichten (oder Dropdowns) werden ausreichend Informationen angezeigt, damit Nutzende entscheiden können, welche Detailansicht geöffnet werden soll.

Die Navigation innerhalb der Anwendung ist sowohl vom Groben ins Detail als auch zurück möglich. Nutzende können beispielsweise von einem Modul zu den zugehörigen Handlungskompetenzen und weiter zur konkreten Handlungskompetenz navigieren. Ebenso ist der Weg von einer Handlungskompetenz zu den zugehörigen Modulen sowie zu den übergeordneten Handlungskompetenzbereichen möglich.

Dadurch werden Zusammenhänge nachvollziehbar dargestellt und ein flexibles Navigieren innerhalb der Datenstruktur ermöglicht. Die Navigation basiert auf den in der API vorhandenen Referenzen zwischen den Objekten.

Ein weiteres Ziel ist eine effiziente Bedienung mit möglichst wenigen Klicks bis zum gewünschten Inhalt. Die Such- und Navigationswege werden kurz gehalten, unter anderem durch direkt auswählbare Dropdown-Resultate, klare Verlinkungen zwischen zusammenhängenden Elementen sowie eine konsistente Detailansicht mit weiterführenden Informationen.

Die Anwendung kommuniziert über HTTP-Requests mit der REST-API und verarbeitet die erhaltenen Daten im Frontend. Dabei werden gängige Angular-Strukturen wie Komponenten, Services und Routing eingesetzt. Zudem werden grundlegende Fehlerbehandlungen sowie verständliche Rückmeldungen bei leeren Suchresultaten berücksichtigt.

Das Ergebnis der Arbeit ist eine funktionsfähige GUI, welche die vorhandenen Bildungsplandaten strukturiert darstellt und durchsuchbar macht. Die entwickelte Oberfläche bildet die Grundlage für zukünftige Erweiterungen und stellt einen Schritt in Richtung einer umfassenderen Applikation zur Unterstützung von Praxisausbildnern und Lernenden dar.

Abnahmekriterien

Die Anwendung gilt als funktional abgenommen, wenn folgende Kriterien erfüllt sind.

1. Exakte Auswahl über Dropdown (Kontext + Objekte)

Für alle folgenden Elemente existiert eine exakte Auswahl über Dropdown (basierend auf List-API-Endpunkten):

- EFZ
- Spezialisierung (falls vorhanden → Auswahl verpflichtend)
- Handlungskompetenzbereiche
- Handlungskompetenzen
- Module

Anforderungen:

- Dropdown-Listen werden über die API geladen.
- Einträge enthalten eindeutige IDs.
- Ein Eintrag kann ausgewählt werden.
- Nach Auswahl werden die korrekten Detaildaten geladen und angezeigt.

Spezifisch für EFZ/Spezialisierung:

- EFZ muss ausgewählt werden.
- Falls Spezialisierungen existieren, muss eine Spezialisierung gewählt werden.
- Ohne vollständige Auswahl:
 - keine Suche möglich
 - keine Detaildaten sichtbar
 - alle angezeigten Daten beziehen sich auf die gewählte Kombination aus EFZ und Spezialisierung.

2. Volltextsuche

- Volltextsuche ist vorhanden.
- Suche erfolgt über (Bezeichnung/Titel und Beschreibung/Details):
 - Handlungskompetenzbereiche
 - Handlungskompetenzen
 - Module
- Resultate beziehen sich auf das gewählte EFZ und die gewählte Spezialisierung.
- Treffer sind anklickbar und führen zur Detailansicht.
- Bei keinem Treffer (kein Suchresultat) wird folgende Meldung angezeigt:

«Ihre Suche lieferte keine Ergebnisse.»

3. Detailansichten

Detailansichten existieren für:

- Handlungskompetenzbereiche
- Handlungskompetenzen
- Module

Verknüpfte Handlungskompetenzbereiche, Handlungskompetenzen und Module werden anhand der in der API vorhandenen Referenzen korrekt nachgeladen und angezeigt, wenn User sich entscheiden, diese anzuzeigen.

Anforderungen:

- Alle von der API gelieferten Informationen werden angezeigt.
- Daten werden korrekt geladen.
- Beziehungen zwischen Objekten sind sichtbar.

4. Navigation zwischen Objekten

Navigation ist möglich zwischen:

- Handlungskompetenzbereich → Handlungskompetenz
- Handlungskompetenz → Modul
- Modul → Handlungskompetenz

- Handlungskompetenz → Handlungskompetenzbereich

Die Navigation funktioniert in beide Richtungen und ohne Fehler. Diese Navigation basiert auf referenzierten IDs.

5. API-Integration

- Daten werden korrekt über die REST-API geladen.
- Angezeigte Daten entsprechen den API-Antworten.
- Beziehungen zwischen HKB, HK und Modulen werden über die vorhandenen Referenzen korrekt aufgelöst.
- Fehlerhafte Requests führen zu verständlicher Rückmeldung.

6. Fehlerbehandlung

- Leere Suchresultate werden angezeigt.
- API-Fehler werden abgefangen.

7. Wichtigste nicht-funktionale Abnahmekriterien

Testing

- Unit-Tests mit Vitest vorhanden.
- Zentrale Komponenten und Services getestet (80% Line-Coverage).
- Tests laufen erfolgreich.

Stabilität

- Anwendung startet fehlerfrei lokal.
- Keine kritischen Konsolenfehler bei Standardnutzung.
- Keine Abstürze bei normaler Nutzung.

Out of Scope (nicht Bestandteil der Abnahme)

Folgende Themen sind bewusst nicht Bestandteil dieser Arbeit:

- Benutzer-Authentifizierung
- Benutzer-Autorisierung
- Rollen- und Berechtigungssystem

Für die vorliegende Datendomäne sind diese aktuell nicht relevant und werden nicht umgesetzt oder bewertet.

Mittel und Methoden

Die Entwicklung der Applikation erfolgt in der Entwicklungsumgebung Visual Studio Code. Für die Umsetzung wird das Framework Angular in der Version 20 eingesetzt. Die Projektverwaltung sowie die Einbindung und Aktualisierung von Abhängigkeiten erfolgen über npm als Package-Manager.

Node.js wird für den Entwicklungs-Workflow und die Nutzung der Angular-CLI verwendet, insbesondere für das Erstellen, Aktualisieren und Ausführen der Anwendung während der Entwicklung. Node.js wird zudem als Runtime-Environment eingesetzt.

Die Benutzeroberfläche wird mit HTML und CSS umgesetzt und in Angular-Komponenten strukturiert. Die Kommunikation mit der bestehenden REST-API erfolgt über HTTP-Requests innerhalb des Angular-Frontends.

Zur Sicherstellung der Codequalität werden automatisierte Tests eingesetzt. Unit-Tests werden mit Vitest durchgeführt, um einzelne Komponenten und Funktionen zu überprüfen. Dadurch wird die Funktionsfähigkeit der Anwendung während der Entwicklung fortlaufend sichergestellt.

Die Aufträge und Tasks werden in Jira in einem Kanban-Board erfasst und bearbeitet, wodurch Transparenz über den Arbeitsstand sowie eine strukturierte Aufgabenorganisation gewährleistet sind.

Der Quellcode wird mit Git versioniert und über GitLab verwaltet. Die Entwicklung erfolgt gemäss dem Git-Workflow, wodurch eine nachvollziehbare Strukturierung von Branches, Commits und Merge-Prozessen sichergestellt ist.

Zur strukturierten Bearbeitung der Aufgaben wird die Projektmethodik IPERKA angewendet, wodurch ein systematisches Vorgehen von der Planung bis zur Umsetzung gewährleistet ist.

Vorkenntnisse

Die Kandidatin verfügt über grundlegende bis gute Kenntnisse in der Entwicklung mit Angular sowie im Umgang mit npm und Node.js. Zudem bestehen gute Kenntnisse in HTML und CSS zur Umsetzung von Benutzeroberflächen.

Weiter verfügt sie über grundlegende Kenntnisse im Umgang mit REST-APIs. Sie kennt die wichtigsten UML-Diagrammtypen und kann Benutzeranforderungen in Form von Use Cases beschreiben.

Zudem verfügt sie über gute Kenntnisse im Umgang mit Figma zur Erstellung von User-Interface-Mockups und zur Visualisierung von Benutzeroberflächen.

Diese Vorkenntnisse bilden die Grundlage für die Umsetzung der Arbeit.

Vorarbeiten

User-Stories werden auf IPA-Start von der verantwortlichen Fachkraft erstellt. Die erforderlichen Daten stehen als Rest-API zur Verfügung.

Neue Lerninhalte

Im Rahmen der Arbeit werden vertiefte Kenntnisse in Angular erworben, insbesondere im Umgang mit Angular Signals zur Zustandsverwaltung und Reaktivität innerhalb der Anwendung.

Zudem wird der Umgang mit dem Angular-CLI-Workflow (Create, Update und weitere CLI-Funktionen) vertieft.

Ein weiterer Schwerpunkt liegt im Bereich Testing. Es werden neue Kenntnisse im Unit-Testing mit Vitest erarbeitet, um einzelne Komponenten und Funktionen systematisch zu testen.

Arbeiten in den letzten 6 Monaten

In den letzten sechs Monaten hat die Kandidatin in einem kleinen Angular-Projekt mitgearbeitet, welches den Ferienerfassungsprozess automatisiert. Dabei sammelte sie praktische Erfahrung in der Umsetzung von Benutzeroberflächen.

Zudem realisierte sie ein kleines Greenfield-Projekt mit Angular v20 zur Anzeige von Daten einer frei verfügbaren Wetter-API. Dabei vertiefte sie ihre Kenntnisse im Aufbau einer Angular-Applikation sowie im Umgang mit REST-APIs.

Zeitraahmen der Aufgabe

Startblock

Startblock 14, KW20: 11.05.2026 – 15.05.2026

PA-Durchführung: 11.05.2026 – 12.06.2026

Einreichung bis: 10.04.2026

Reserve-Startblock für Inf. API

Schultag / BMS: Freitag

Durchführung von 15.05.2026 00:00 bis 04.06.2026 18:00

Bemerkungen

z.B. Militärdienst, Stellvertretung etc.

Bitte beachten Sie, dass der 26. und der 27. Mai ebenfalls IPA-Tage sind. Frau Schäfer hat einen Nachteilsausgleich von zwei Tagen, diese zusätzlichen Tage kann man jedoch nicht als IPA-Tage im Tool erfassen. Auf Ratschlag von Herrn Luca Lindegger habe ich diese beiden Tage als "Andere" ausgewählt.